# LAPORAN PRAKTIKUM ALGORITMA DAN PEMROGRAMAN 2

## MODUL 5

## “REKURSIF”

**DISUSUN OLEH:**

**DYAH IMANSARI**

**109082500130**

**S1IF-13-02**

**DOSEN:**

**Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2025/2026**

# DASAR TEORI

1. **Definisi Rekursif**

   Rekursif adalah sebuah teknik di dalam pemrograman di mana sebuah subprogram (fungsi atau prosedur) memanggil dirinya sendiri. Berbeda dengan iterasi (perulangan biasa), rekursif memecah masalah besar menjadi sub-masalah yang lebih kecil namun memiliki tipe yang identik. Proses ini terus berlanjut hingga mencapai suatu kondisi yang sangat sederhana yang bisa langsung diselesaikan.

2. **Komponen Utama Algoritma Rekursif**

   Setiap algoritma rekursif yang benar harus memiliki dua komponen wajib untuk menghindari perulangan tanpa henti (*infinite recursion*):

   - **Basis (Base Case):** Bagian yang berisi nilai penyelesaian yang sudah pasti dan tidak memerlukan pemanggilan diri sendiri lagi. Basis berfungsi sebagai instruksi untuk menghentikan proses rekursif.
   - **Rekurens (Recursive Case):** Bagian yang mendefinisikan masalah utama ke dalam bentuk sub-masalah yang lebih kecil. Di bagian ini, parameter yang dikirim harus selalu bergerak maju menuju kondisi basis agar program bisa berhenti.

3. **Proses Eksekusi: Forward dan Backward**

   Dalam eksekusi program rekursif, terdapat dua fase penting yang terjadi di dalam memori komputer:

   - **Forward (Maju):** Fase di mana subprogram terus-menerus memanggil dirinya sendiri karena kondisi basis belum tercapai. Pada fase ini, instruksi-instruksi yang belum selesai akan "ditumpuk" terlebih dahulu.
   - **Backward (Mundur):** Fase yang dimulai setelah kondisi basis terpenuhi. Program akan kembali ke titik pemanggilan sebelumnya untuk menyelesaikan instruksi-instruksi yang tadi tertunda, dimulai dari tumpukan yang paling akhir hingga kembali ke pemanggilan awal.

# LATIHAN

1. **Latihan 1**

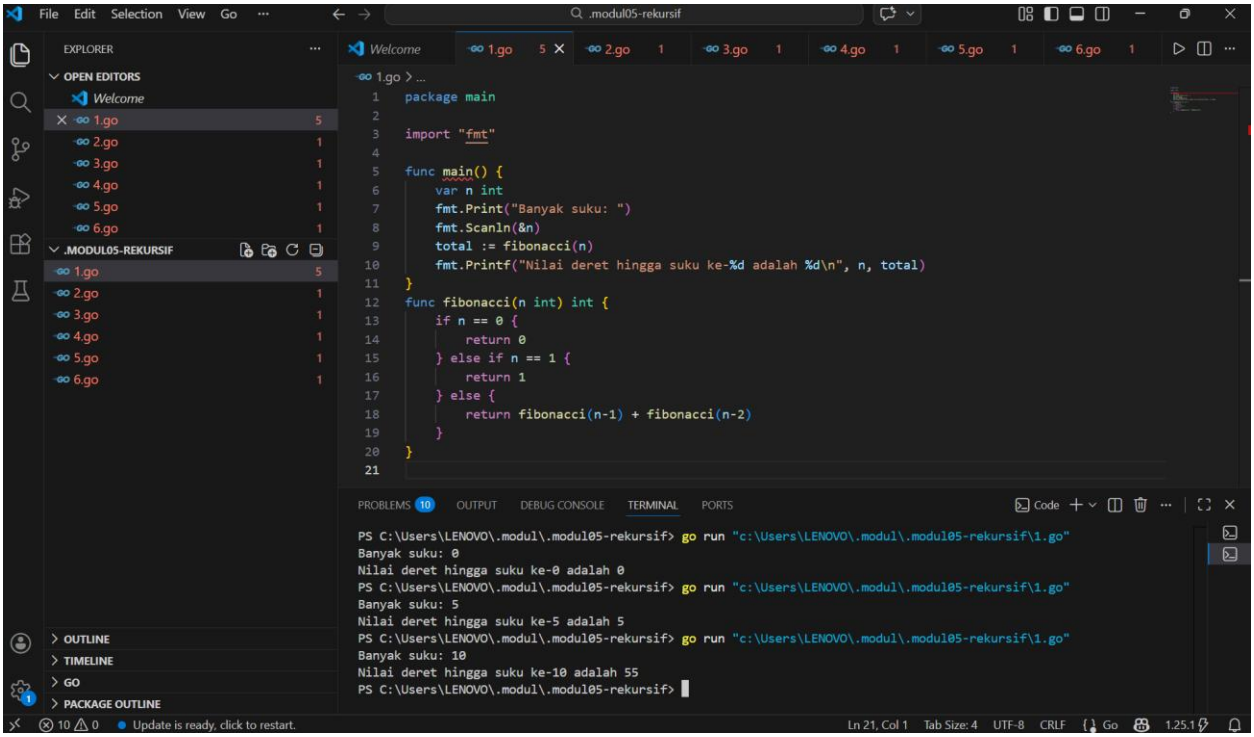
**Source Code**

```
package main

import "fmt"

func main() {
    var n int
    fmt.Print("Banyak suku: ")
    fmt.Scanln(&n)
    total := fibonacci(n)
    fmt.Printf("Nilai deret hingga suku ke-%d adalah
%d\n", n, total)
}
func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}
```

**Screenshoot program**




**Deskripsi program**

Program ini bertujuan untuk menghitung nilai deret hingga suku ke-n pada deret fibonacci. Penggunaan rekursif dalam program ini menyelesaikan masalah dengan memecah satu nilai suku menjadi penjumlahan suku-suku yang lebih kecil hingga mencapai nilai dasar yang telah ditentukan.

Program dimulai dengan meminta input nilai n dari pengguna melalui program utama. Fungsi rekursif fibonacci(n) kemudian bekerja dengan dua basis (titik henti): jika n = 0, fungsi akan mengembalikan nilai 0, dan jika n = 1, fungsi akan mengembalikan nilai 1. Apabila n lebih besar dari 1, fungsi akan masuk ke bagian rekurens dengan menjumlahkan dua hasil pemanggilan dirinya sendiri, yaitu fibonacci(n-1) + fibonacci(n-2). Proses ini terus berlanjut hingga mencapai basis, lalu hasil akhirnya dijumlahkan kembali dan ditampilkan ke layar sebagai nilai deret suku ke-n.

2. **Latihan 2**

**Source Code**

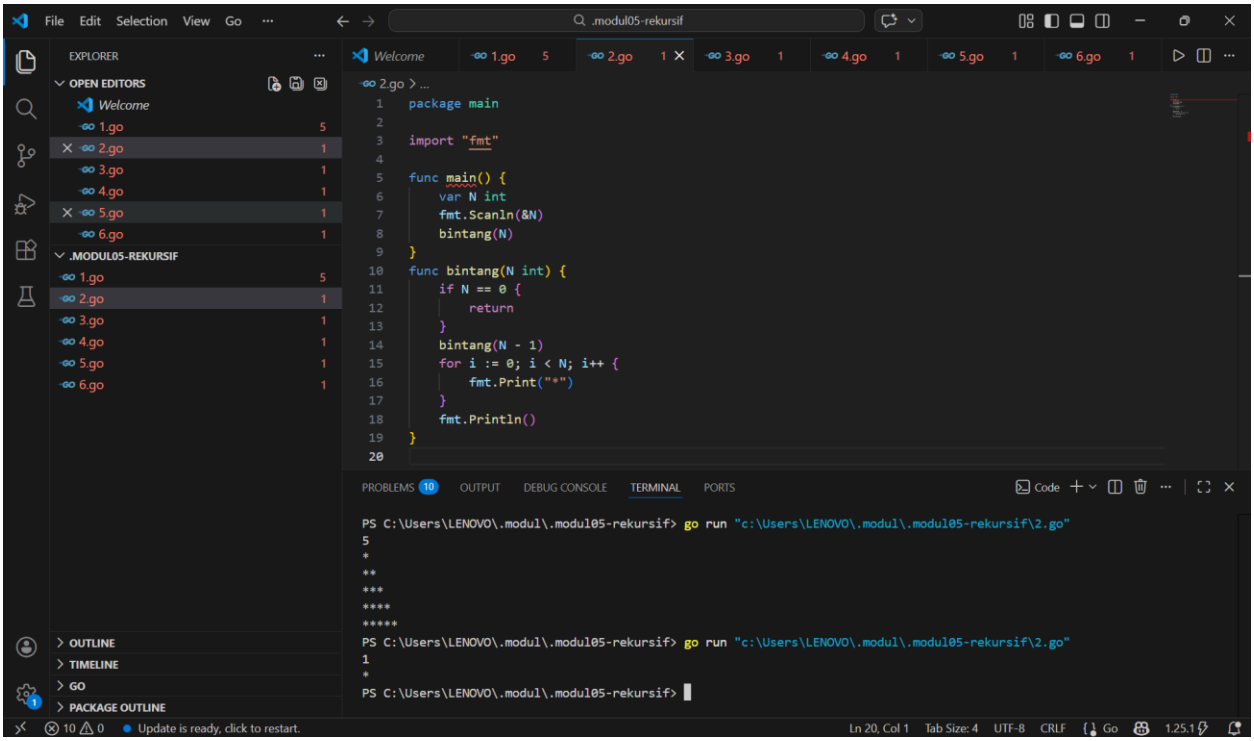
```
package main

import "fmt"

func main() {
    var N int
    fmt.Scanln(&N)
```

```
        bintang(N)
    }
    func bintang(N int) {
        if N == 0 {
            return
        }
        bintang(N - 1)
        for i := 0; i < N; i++ {
            fmt.Print("*")
        }
        fmt.Println()
    }
```

**Screenshoot program**




**Deskripsi program**

Program ini dirancang untuk menampilkan pola karakter bintang (*) yang membentuk segitiga siku-siku berdasarkan jumlah baris yang diinputkan oleh pengguna.

Cara kerjanya, program menerima input nilai n yang menentukan tinggi segitiga. Fungsi rekursif akan mengecek basis, yaitu jika nilai baris saat ini sudah melebihi n, maka proses

berhenti. Jika belum, fungsi akan mencetak karakter bintang (*) sebanyak nomor baris tersebut, kemudian melakukan pemanggilan rekursif untuk baris berikutnya (baris + 1) hingga pola terbentuk sempurna dari atas ke bawah.

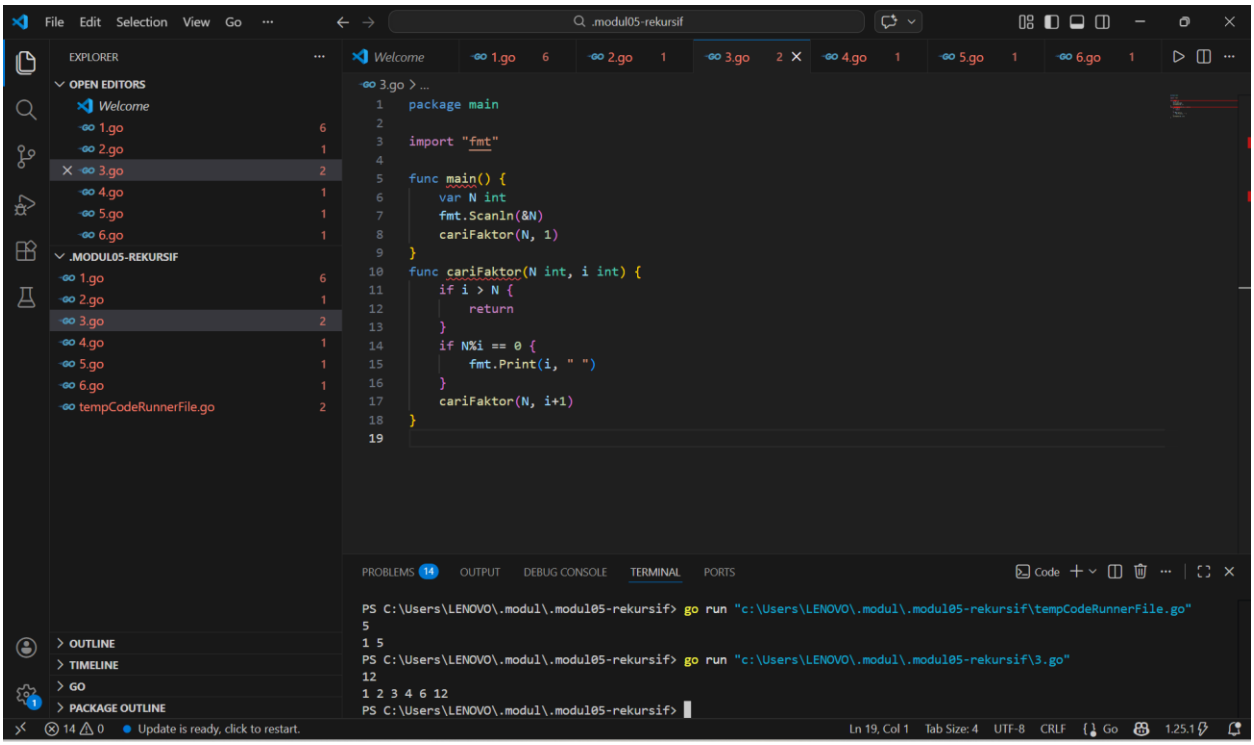
3. **Latihan 3**
   **Source Code**

```
package main

import "fmt"

func main() {
    var N int
    fmt.Scanln(&N)
    cariFaktor(N, 1)
}
func cariFaktor(N int, i int) {
    if i > N {
        return
    }
    if N%i == 0 {
        fmt.Print(i, " ")
    }
    cariFaktor(N, i+1)
}
```

**Screenshoot program**




**Deskripsi program**

Program ini mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N. Program ini memanfaatkan sebuah subprogram (fungsi) bernama cariFaktor(N, i) untuk melakukan pengecekan angka pembagi secara berurutan. Dengan menggunakan metode rekursif, proses iterasi untuk memeriksa setiap angka dari 1 hingga N dilakukan melalui pemanggilan fungsi itu sendiri, yang memberikan struktur kode yang lebih modular dan sistematis.

Cara kerjanya, program akan mengecek setiap nilai i apakah dapat membagi habis N tanpa sisa (menggunakan operasi %). Jika iya, maka nilai i tersebut dicetak ke layar sebagai faktor dari bilangan tersebut. Setelah itu, fungsi akan memanggil dirinya sendiri kembali dengan parameter i + 1 untuk memeriksa angka berikutnya. Proses ini akan terus berulang hingga mencapai basis, yaitu kondisi di mana angka pembagi i sudah melebihi nilai N, sehingga program berhenti secara otomatis setelah semua faktor ditemukan dan ditampilkan secara berurutan.

4. **Latihan 4**

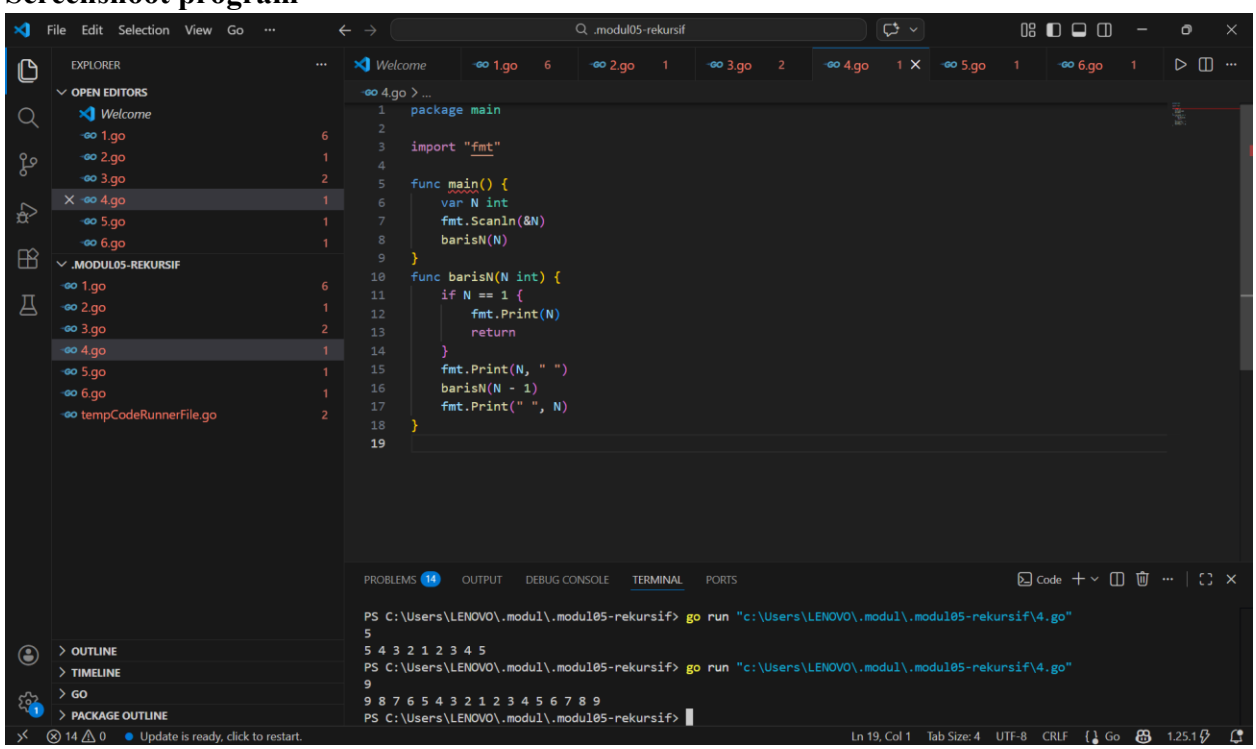
**Source Code**

```
package main


import "fmt"
```

```go
func main() {
    var N int
    fmt.Scanln(&N)
    barisN(N)
}
func barisN(N int) {
    if N == 1 {
        fmt.Print(N)
        return
    }
    fmt.Print(N, " ")
    barisN(N - 1)
    fmt.Print(" ", N)
}
```

**Screenshoot program**




**Deskripsi program**

Program ini bertujuan untuk menampilkan barisan bilangan yang berpola menurun dari N hingga 1 dan kembali meningkat hingga N. Program ini memanfaatkan subprogram bernama barisN(N) untuk menangani proses cetak angka secara sistematis.

Cara kerjanya, program pertama-tama akan mencetak nilai N saat ini ke layar, kemudian melakukan pemanggilan dirinya sendiri dengan parameter N - 1. Proses pengurangan ini akan terus berulang hingga mencapai basis, yaitu kondisi di mana nilai N sama dengan 1. Setelah mencapai basis dan mencetak angka 1, fungsi-fungsi yang tertunda di dalam *stack* akan mulai diselesaikan, yang kemudian menjalankan instruksi cetak kedua untuk menampilkan kembali nilai N dari kecil ke besar Program akan berhenti secara otomatis setelah barisan bilangan berpola menurun dari N hingga 1 dan kembali meningkat hingga N sesuai dengan input yang diberikan.

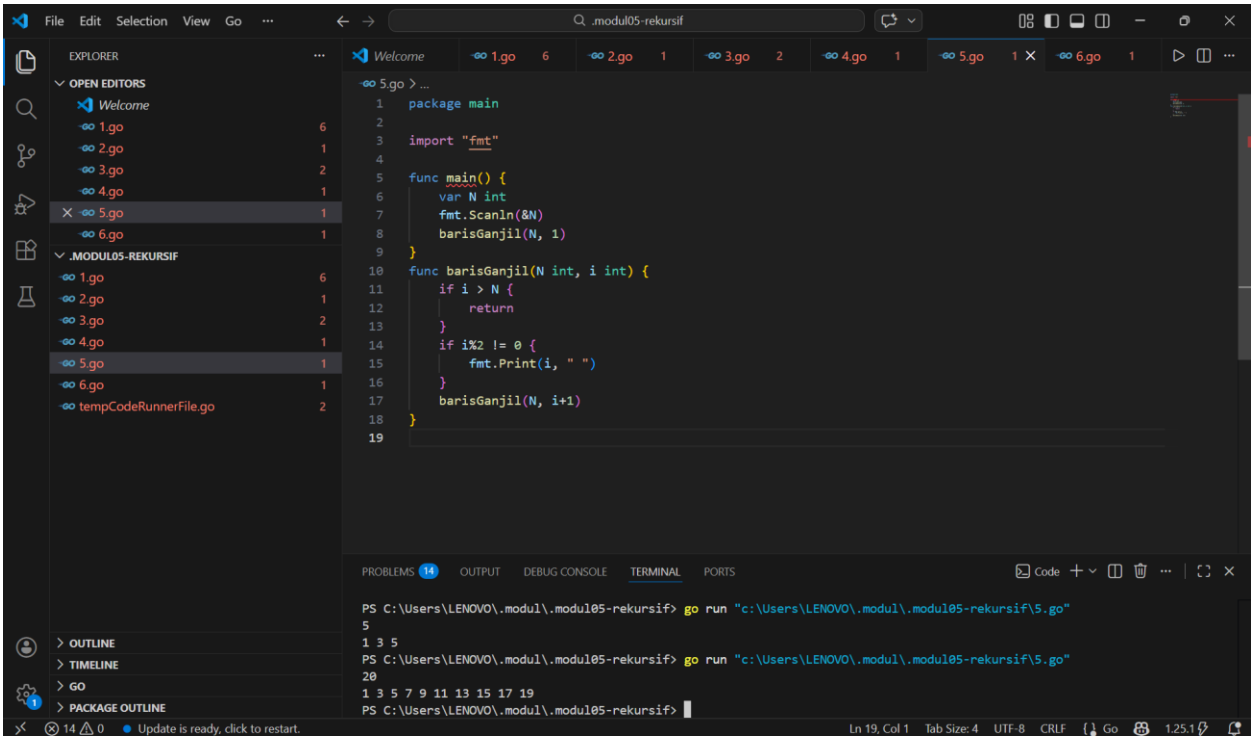
5. **Latihan 5**
   **Source Code**

```
package main

import "fmt"

func main() {
    var N int
    fmt.Scanln(&N)
    barisGanjil(N, 1)
}
func barisGanjil(N int, i int) {
    if i > N {
        return
    }
    if i%2 != 0 {
        fmt.Print(i, " ")
    }
    barisGanjil(N, i+1)
}
```

**Screenshoot program**




**Deskripsi program**

Program ini dirancang untuk mencari dan menampilkan barisan bilangan ganjil dari 1 hingga batas nilai N. Program ini memanfaatkan sebuah subprogram bernama barisGanjil(N, i) untuk melakukan pengecekan angka secara berurutan mulai dari angka pertama. Dengan menerapkan metode rekursif, proses pemeriksaan setiap bilangan dilakukan melalui pemanggilan fungsi itu sendiri secara berulang.

Cara kerjanya, program akan mengecek setiap nilai i apakah termasuk dalam kategori bilangan ganjil dengan menggunakan operasi sisa bagi (i % 2 != 0). Jika kondisi tersebut terpenuhi, maka nilai i akan dicetak ke layar sebagai bagian dari barisan. Setelah proses pengecekan satu angka selesai, fungsi akan memanggil dirinya sendiri dengan menaikkan nilai parameter menjadi i + 1 untuk memeriksa angka selanjutnya. Proses ini akan terus berjalan hingga mencapai basis, yaitu kondisi dimana nilai i sudah melebihi batas N, sehingga program akan berhenti secara otomatis setelah seluruh barisan bilangan ganjil ditampilkan.

6. **Latihan 6**

**Source Code**

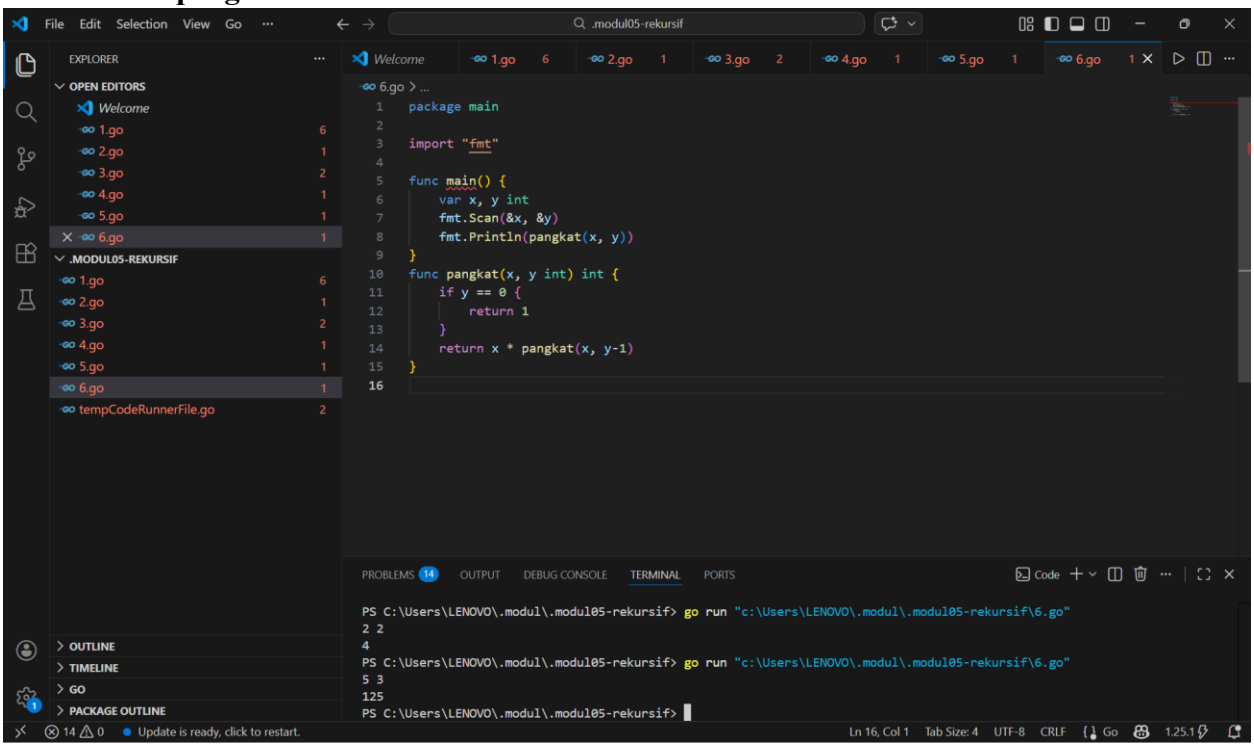
```
package main

import "fmt"

func main() {
```

```
        var x, y int
        fmt.Scan(&x, &y)
        fmt.Println(pangkat(x, y))
    }
    func pangkat(x, y int) int {
        if y == 0 {
            return 1
        }
        return x * pangkat(x, y-1)
    }
```

**Screenshoot program**




**Deskripsi program**

Program ini mengimplementasikan rekursif untuk mencari hasil pangkat dari dua buah bilangan, yaitu basis x dan eksponen y. Program ini memanfaatkan sebuah subprogram bernama pangkat(x, y) untuk melakukan perkalian berulang.

Cara kerjanya, program akan terus mengalikan nilai x dengan hasil pemanggilan fungsi dirinya sendiri yang parameter eksponennya telah dikurangi satu (y - 1). Proses perkalian

beruntun ini akan terus berjalan di dalam hingga mencapai basis, yaitu kondisi dimana nilai eksponen y sama dengan 0. Karena bilangan apapun yang dipangkatkan 0 hasilnya adalah 1, maka fungsi akan mulai mengembalikan nilai tersebut dan menyelesaikan seluruh perkalian hingga diperoleh hasil akhir. Program secara otomatis akan berhenti dan menampilkan hasil perpangkatan setelah semua proses rekursi selesai dieksekusi.

# DAFTAR PUSTAKA

Tim Penyusun. (2026). *Modul Praktikum Algoritma dan Pemrograman 2*. Bandung: Fakultas Informatika, Telkom University.